03장 프로세스

학습목표

- 프로세스 생성과 상태 변화를 이해한다.
- 프로세스 제어 블록의 구성과 문맥 교환 과정을 학습한다.
- 프로세스 복사 및 전환으로 만들어지는 프로세스 계층구조를 살펴본다.

목차

1. 프로세스의 개념과 상태
2. 프로세스 제어 블록과 문맥 교환
3. 프로세스 계층구조
4. 심화 프로세스의 데이터 영역

1. 프로세스의 개념

◆프로그램과 프로세스

- 프로세스: 프로그램이 실행되어 메모리에 올라온 상태
- 폰노이만 구조에서 프로그램 실행은
  프로그램의 코드가 메모리에 올라와 프로세스가 되는 것을 의미함

그림 3-1 프로그램과 프로세스

1. 프로세스의 개념

◆프로그램과 프로세스

- 윈도우 운영체제는 작업 관리자의 [프로세스] 탭에서 실행 중인 모든 프로세스 확인
- 커널 프로세스와 사용자 프로세스

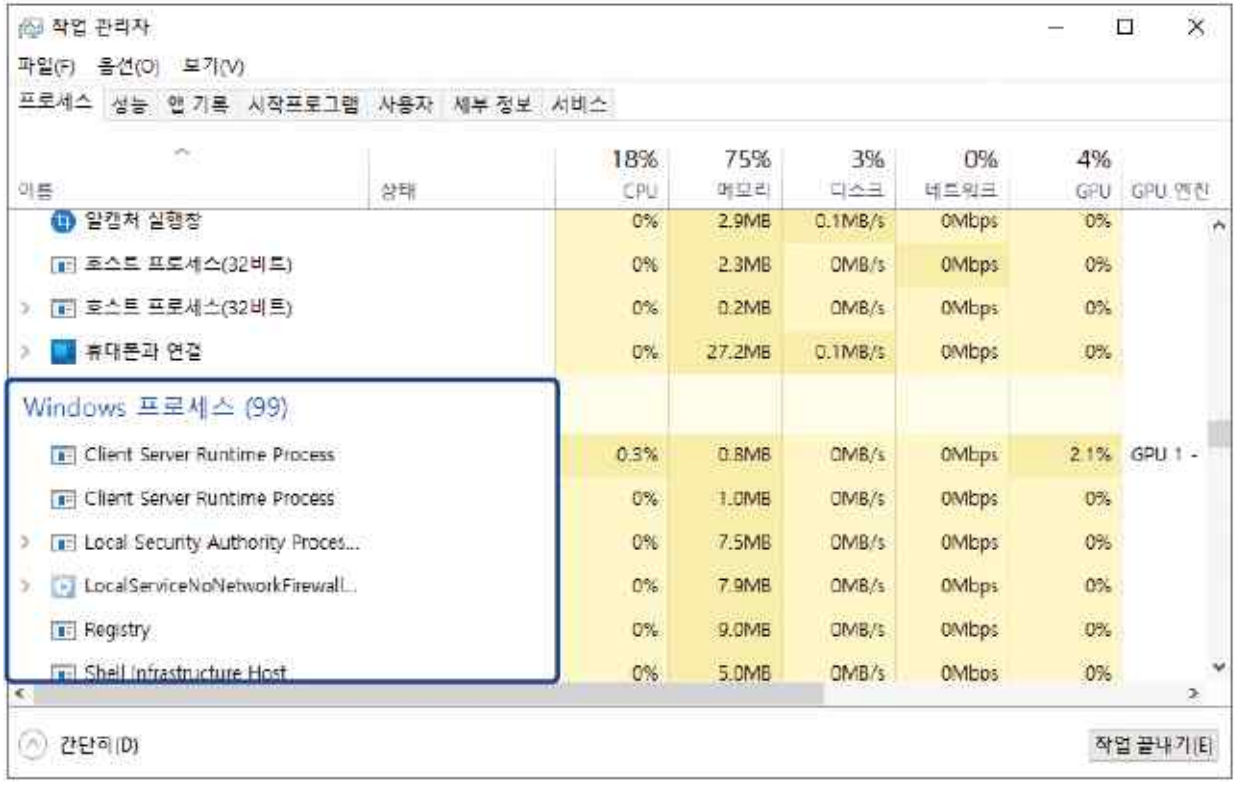


그림 3-2 작업 관리자와 프로세스

1. 프로세스의 개념

◆ 프로세스의 구조

- 코드 영역과 데이터 영역
- 정적 데이터 영역, 힙 영역, 스택 영역

그림 3-3 프로세스의 구조

2. 프로세스의 상태

◆요리사 모형으로 프로세스의 상태 이해하기

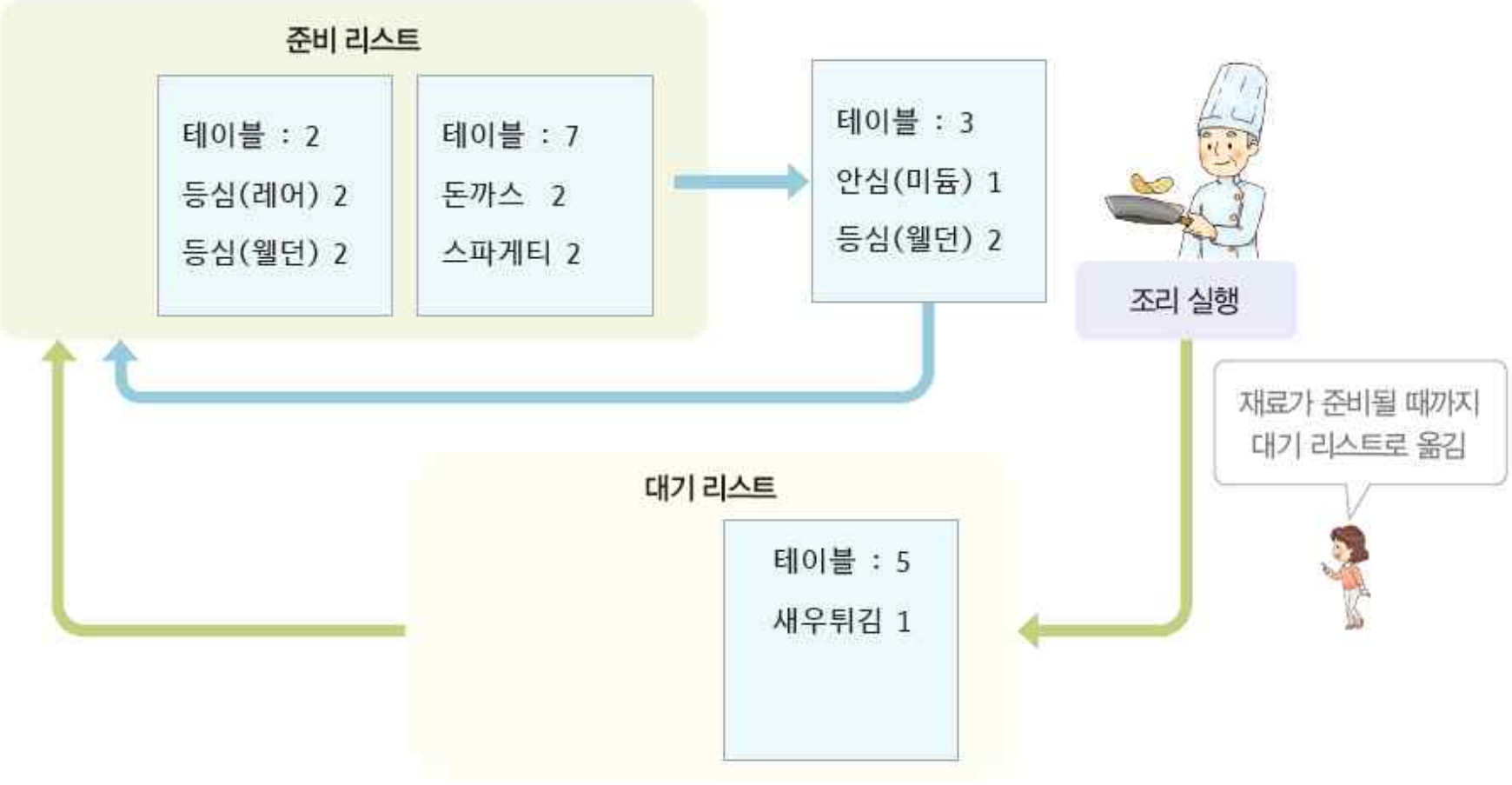


그림 3-4 요리 준비와 실행

2. 프로세스의 상태

◆요리사 모형으로 프로세스의 상태 이해하기

- 프로세스가 되면 운영체제로부터 프로세스 제어 블록(PCB)을 받는다.
- 프로그램 + PCB = 프로세스
  프로세스 – PCB = 프로그램

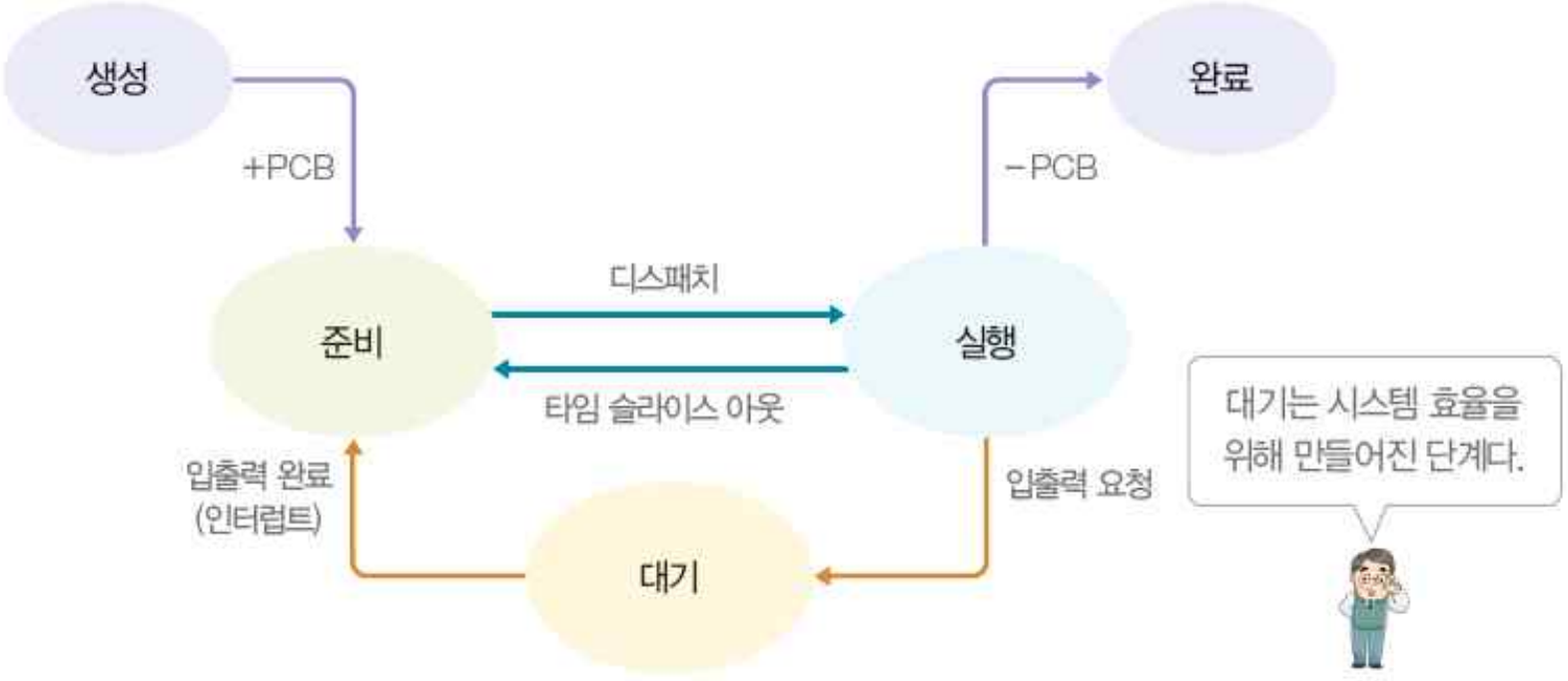


그림 3-5 프로세스 상태 전이도

2. 프로세스의 상태

◆요리사 모형으로 프로세스의 상태 이해하기

- **생성 상태**
  - 프로그램이 메모리에 올라와 프로세스가 생성된 상태
- **준비 상태**
  - 프로세스에 CPU가 할당되기를 기다리는 상태
- **실행 상태**
  - CPU를 할당받아 실제 작업을 수행하는 상태
- **대기 상태**
  - 실행 중인 프로세스가 입출력 요청 후 기다리는 상태
- **완료 상태**
  - 프로세스가 모든 작업을 완료한 상태

2. 프로세스의 상태

◆프로세스의 비활성 상태

- 휴식 상태: 프로세스가 메모리에 있으나 일시적으로 실행을 멈춘 상태
  - 재시작

```
$ sleep 100
^Z
[1]+ Stopped        sleep 100
$ jobs
[1]+ Stopped        sleep 100
$ bg
[1]+ sleep 100 &
$ jobs
[1]+ Running        sleep 100 &
```

그림 3-6 유닉스에서의 프로세스 휴식 상태

2. 프로세스의 상태

◆프로세스의 비활성 상태

- 보류 상태: 프로세스가 메모리에서 쫓겨난 상태
  - 컴퓨터 성능을 떨어뜨리거나 실행을 미루어도 지장이 없는 프로세스

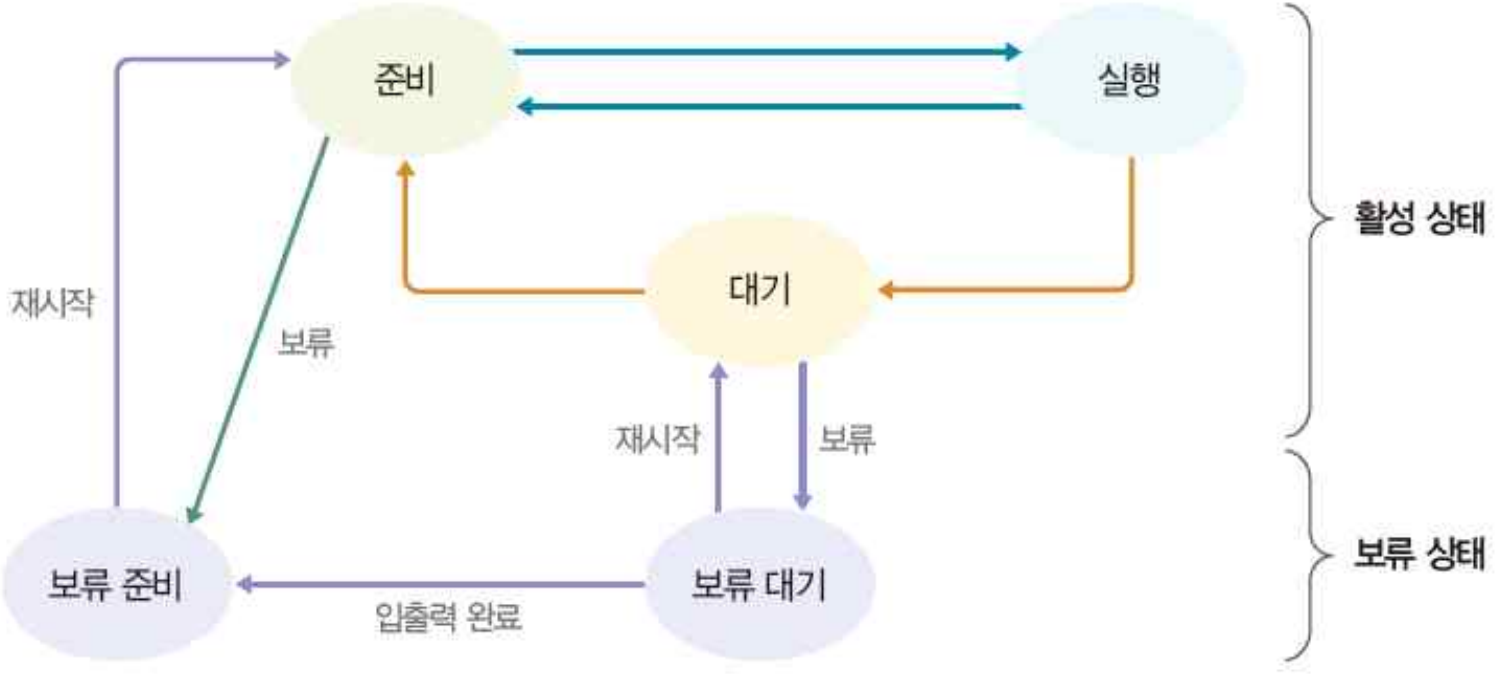


그림 3-7 보류 상태를 포함한 프로세스 상태 전이도

Thank you!

1. 프로세스 제어 블록

◆프로세스 제어 블록의 필요성

- 주문서처럼, 프로세스가 실행되는 동안 작업 처리에 필요한 정보를 PCB에 저장

그림 3-8 주문서와 조리 작업

1. 프로세스 제어 블록

◆프로세스 제어 블록의 필요성

- 멀티태스킹에서 CPU를 다시 할당받았을 때 이전의 작업 상태를 저장하고 다시 불러오기 위해 PCB에 관련 정보를 기록

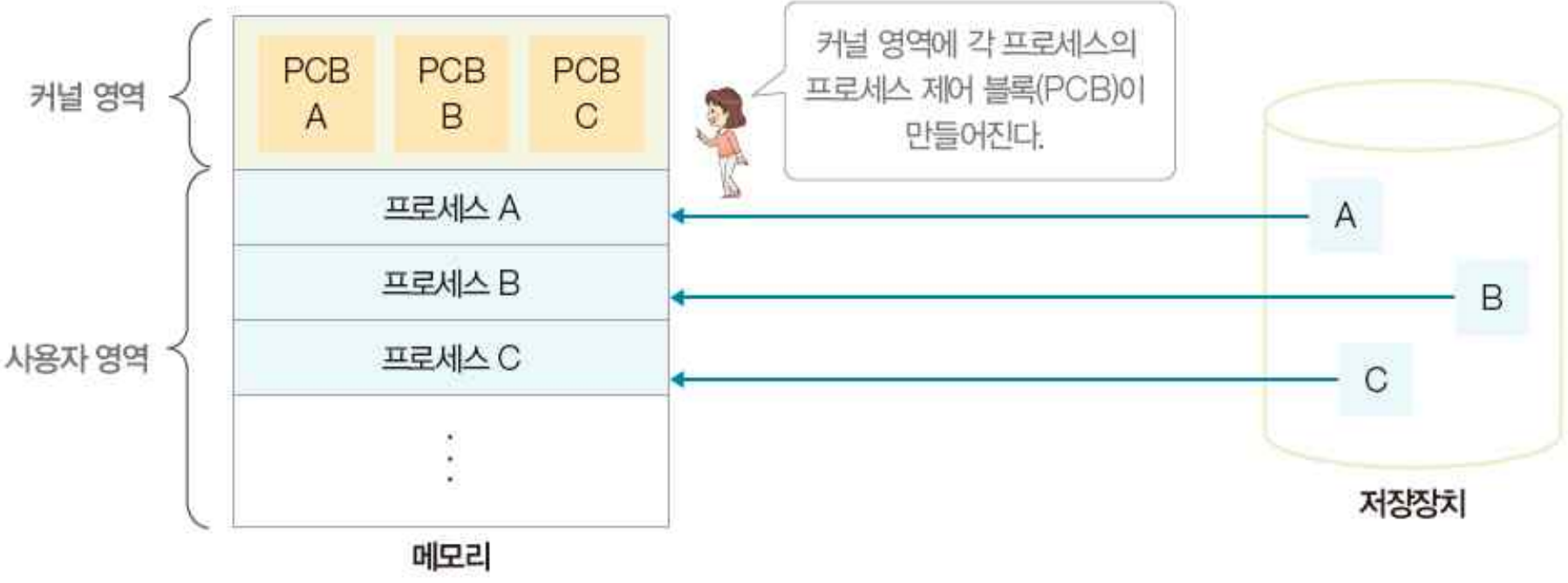


그림 3-9 프로세스와 PCB의 관계

1. 프로세스 제어 블록

◆프로세스 제어 블록의 구성 요소

PCB가 큐에 삽입될 때 포인터가 필요해.

포인터(pointer)	프로세스 상태
PID (프로세스 구분자)	
PC (프로그램 카운터)	
프로세스 우선순위	
각종 레지스터 정보	
메모리 관리 정보	
할당된 자원 정보	
계정(account) 정보	
PPID(부모 PID)와 CPID(자녀 PID)	
⋮	

프로세스 상태와 PID가 앞쪽에 나와.

그림 3-10 프로세스 제어 블록의 구성

2. 문맥 교환

◆문맥 교환의 개념

- 조리 작업 전환을 위해서는 주문서 교환은 물론 작업 환경도 바꾸어야 함

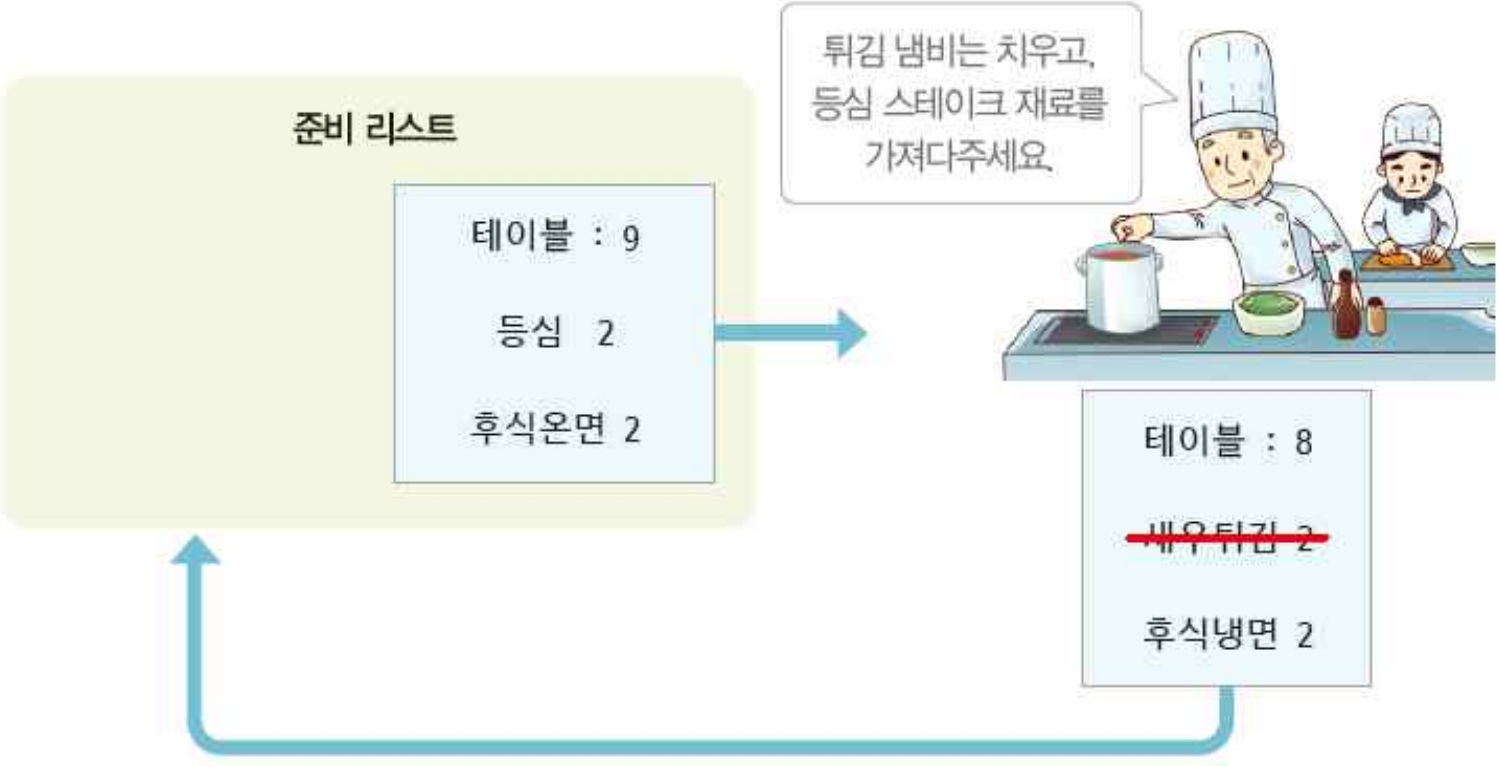


그림 3-11 주문서 교환

2. 문맥 교환

◆문맥 교환의 개념

- 문맥교환: CPU를 사용하던 프로세스를 중단하고 새로운 프로세스로 전환하는 과정
- 타임 슬라이스 아웃

2. 문맥 교환

◆문맥 교환 과정

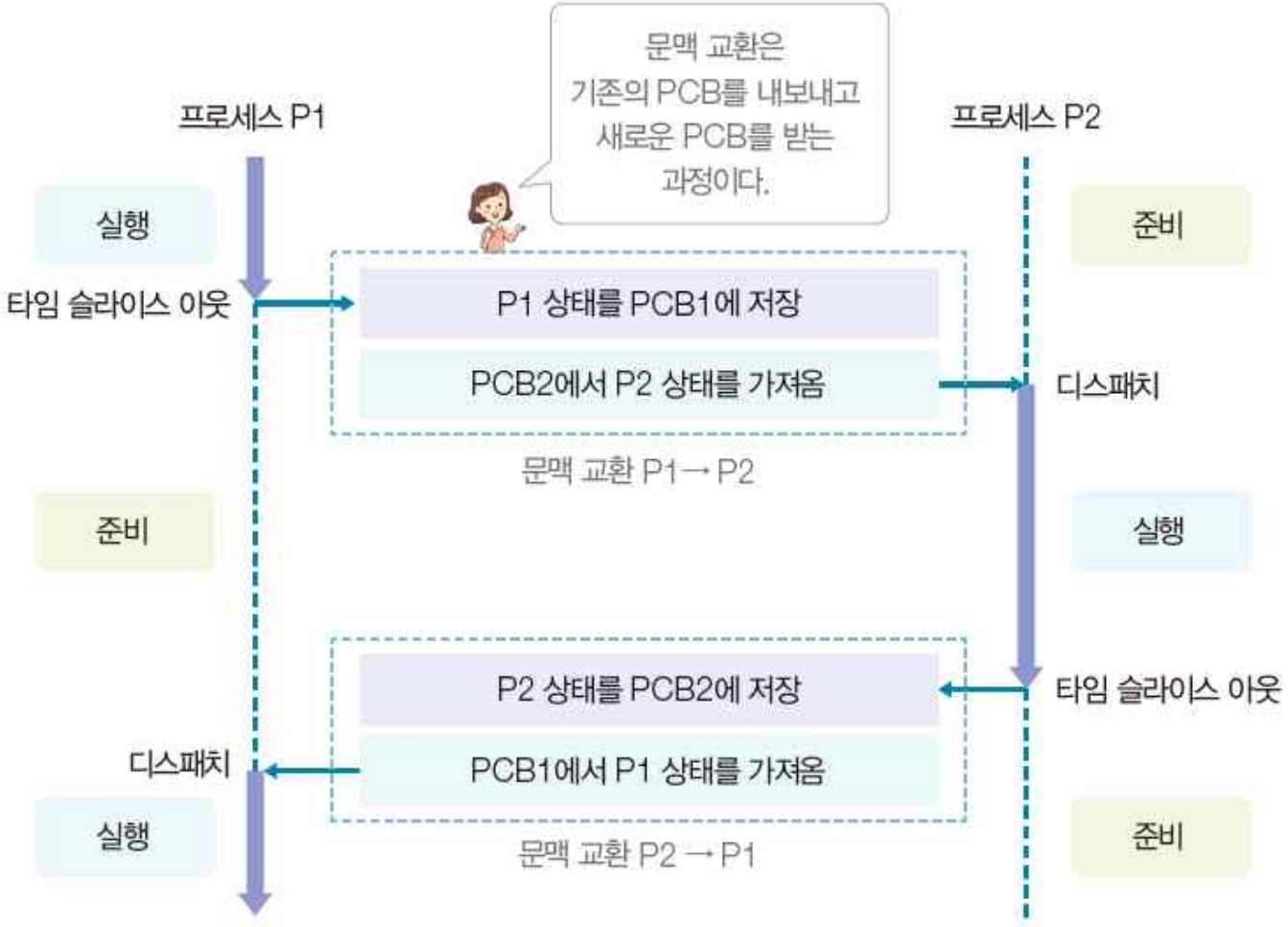


그림 3-12 문맥 교환 과정

2. 문맥 교환

◆문맥 교환과 타임 슬라이스의 크기

- 타임 슬라이스가 너무 크면 하나의 프로세스가 오래 실행되어 다른 프로세스의 대기 시간이 길어짐
- 타임 슬라이스가 너무 작으면 작업 시간보다 문맥 교환 시간이 커져 성능이 저하됨

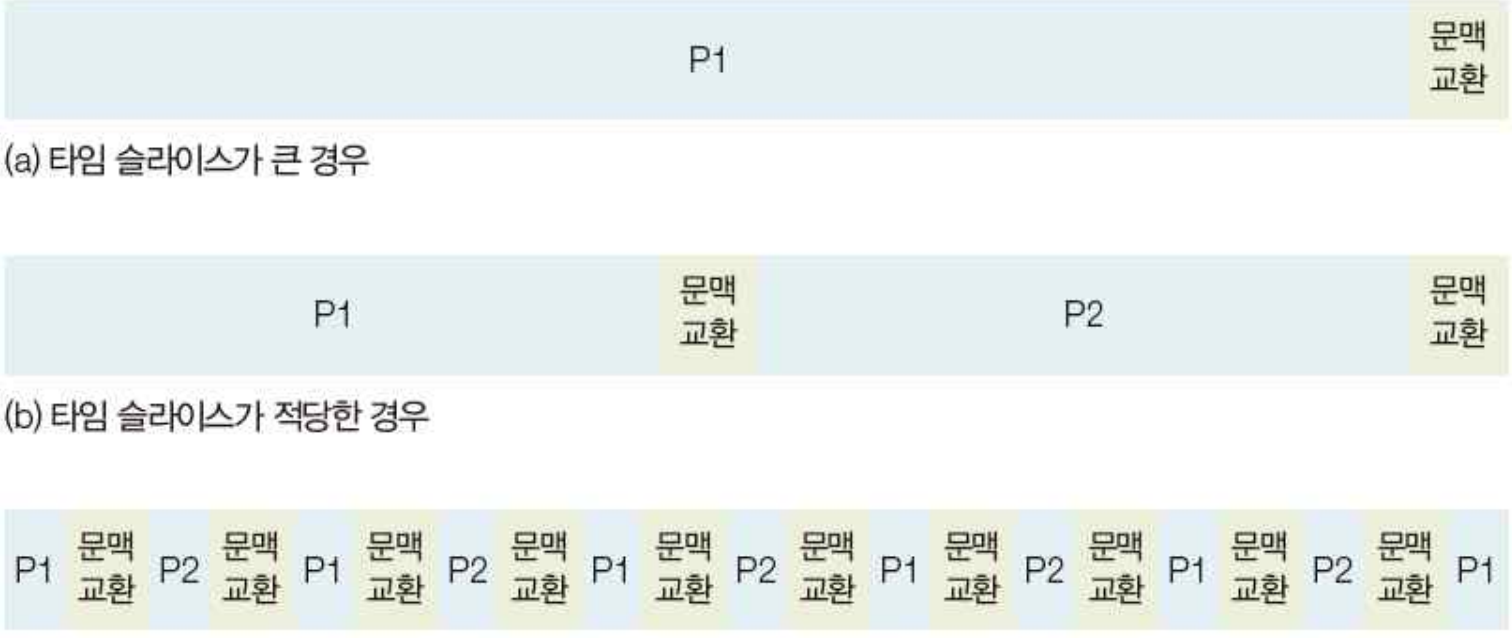


그림 3-13 타임 슬라이스의 크기와 문맥 교환

1. 프로세스 계층구조의 필요성

◆가비지 컬렉션

- 가비지 컬렉션: 불필요한 프로세스를 정리하고 자원을 회수하는 작업
- 컴퓨터를 켜놓고 오래 사용하다 보면 점점 느려짐
  - 프로세스가 종료되지 않고 계속 메모리에 남아 있으면 자원이 낭비되기 때문
- 컴퓨터를 껐다가 다시 시작하면 시스템이 원래 속도로 돌아옴
  - 모든 프로세스가 강제 종료되어 자원을 완전히 회수한 후 다시 시작하기 때문
- 효율적인 자원 회수를 위해 서로 관련 있는 프로세스를 부모-자식 관계로 묶어 관리
  - 부모 프로세스가 자식 프로세스를 생성
  - 자식 프로세스가 종료되면 부모 프로세스가 자식 프로세스의 자원을 회수

1. 프로세스 계층구조의 필요성

◆가비지 컬렉션

- 모든 프로세스는 부모-자식 관계를 형성함
  - 트리 모양의 계층구조

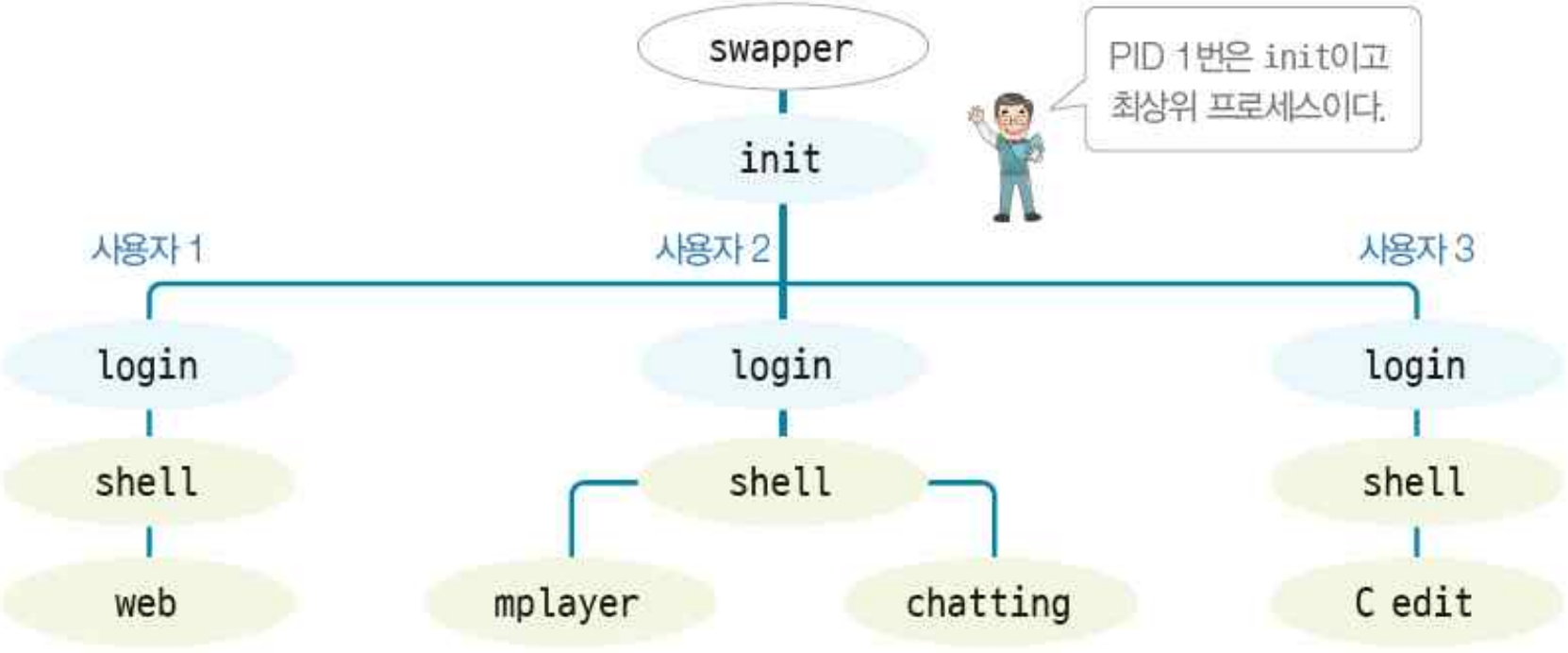


그림 3-14 유닉스 운영체제의 프로세스 계층구조

2. 프로세스 생성 시스템 호출

◆자식 프로세스를 생성하는 fork()

- 기존의 프로세스로부터 자식 프로세스를 생성
- 구글 크롬 브라우저에서 [Ctrl]+[N]으로 새 창 열기
  - 메모리에서 기존 프로세스를 복사하기 때문에 빠름
- 다중 인스턴스: 같은 프로세스를 여러 개 만들어 실행

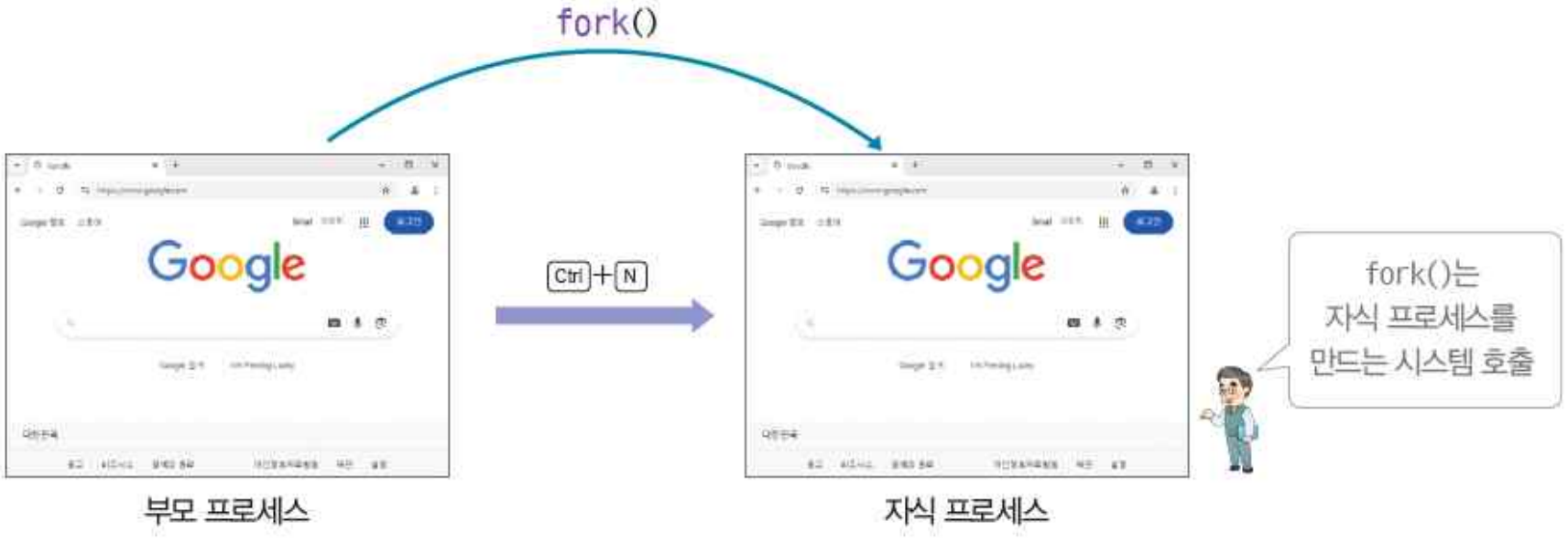


그림 3-15 fork() 시스템 호출

2. 프로세스 생성 시스템 호출

◆자식 프로세스를 생성하는 fork()

- fork() 시스템 호출의 장점
  - 병렬 처리 구현, 빠른 프로세스 생성, 손쉬운 자원 상속, 편리한 자원 회수

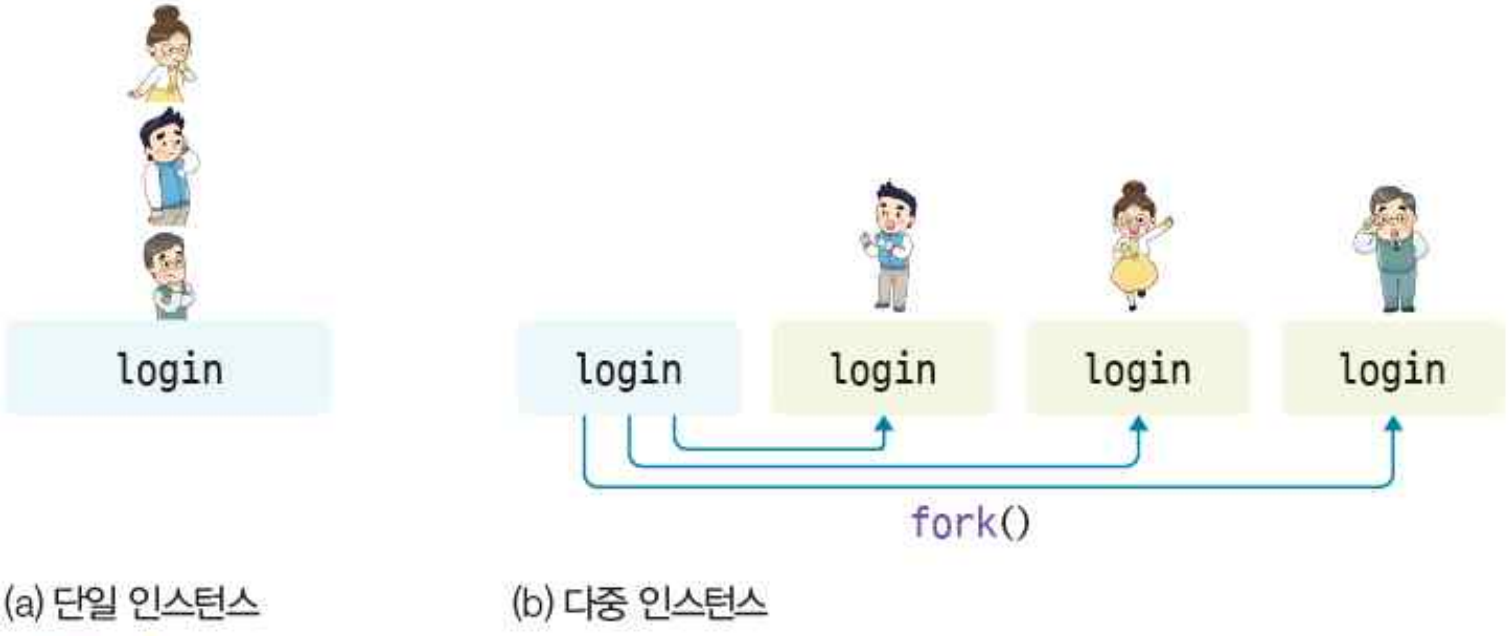


그림 3-16 fork()와 다중 인스턴스

2. 프로세스 생성 시스템 호출

◆자식 프로세스를 생성하는 fork()

- 부모 프로세스(pd>0)과 자식 프로세스(pd==0)은 서로 독립적인 프로세스
- 실행 순서는 알 수 없으나, 부모는 자식 종료를 기다렸다가 자원을 회수해야 함
- 따라서 부모 프로세스는 wati() 함수로 자식 프로세스를 기다림

03 프로세스 계층구조

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main( ) {
   int pd;

   pd = fork();

   if (pd = 0) {
      printf("Child %d\n", getpid());
   } else if (pd > 0) {
      wait(NULL);
      printf("Parent %d\n", getpid());
   } else {   // pid = -1
      printf("Fork failed\n");
   }
   return 0;
}
```

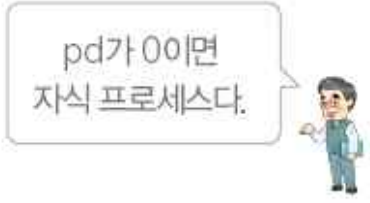
부모 프로세스

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main( ) {
   int pd;

   pd = fork();

   if (pd = 0) {
      printf("Child %d\n", getpid());
   } else if (pd > 0) {
      wait(NULL);
      printf("Parent %d\n", getpid());
   } else {   // pid = -1
      printf("Fork failed\n");
   }
   return 0;
}
```



자식 프로세스

그림 3-17 fork() 작동 사례

2. 프로세스 생성 시스템 호출

◆자식 프로세스를 생성하는 fork()

- fork() 실행 이후 부모 프로세스와 자식 프로세스에서 달라지는 정보
  - 메모리 정보, PID, PPID, CPID

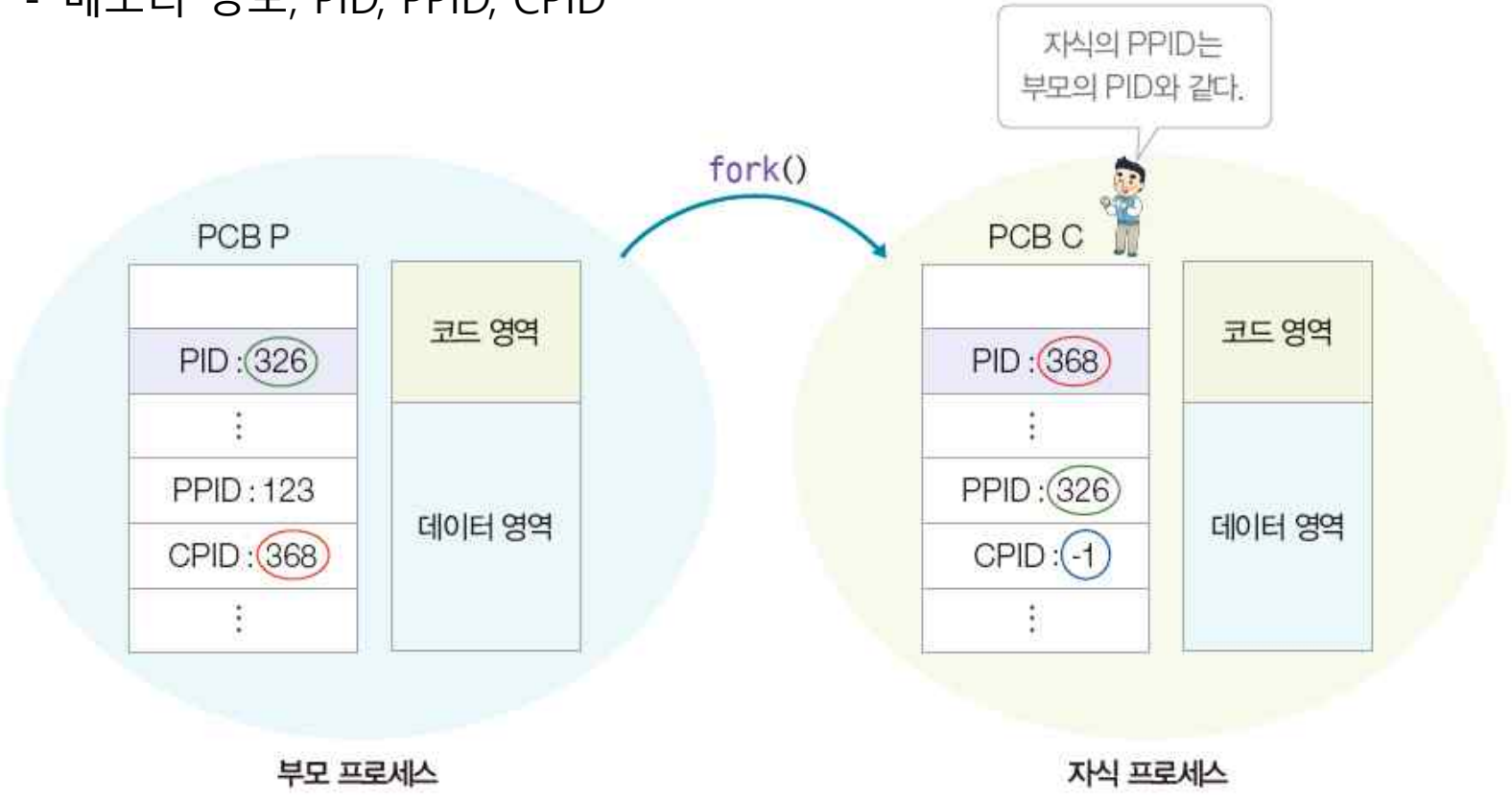
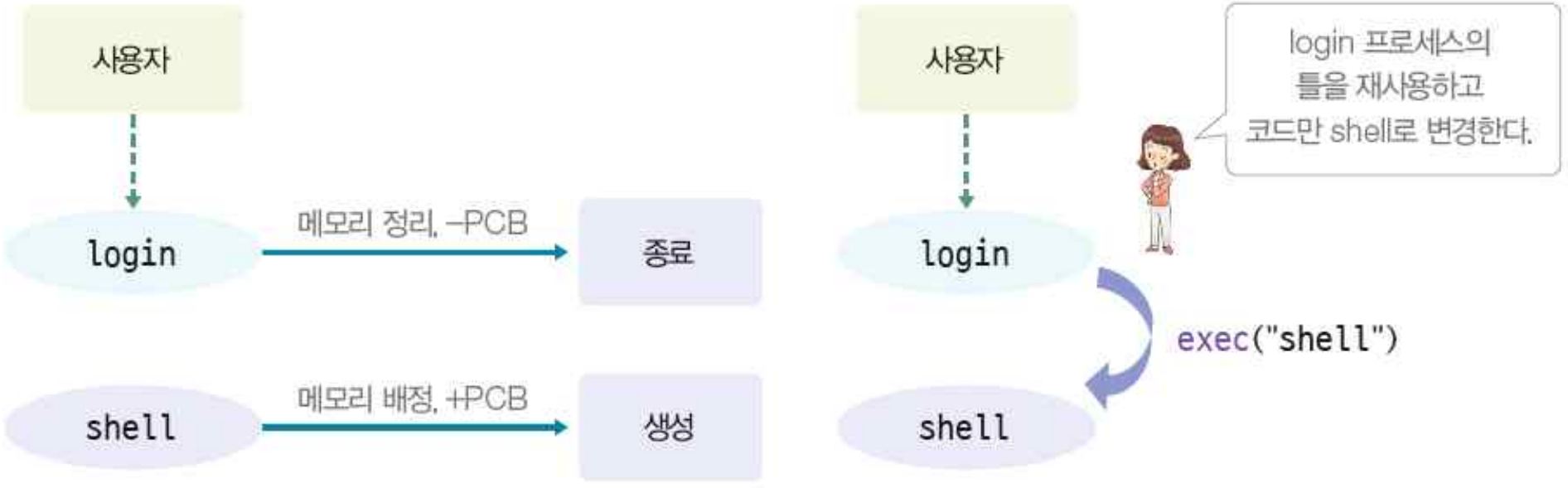


그림 3-18 fork() 호출 후의 PCB

3. 프로세스 재사용 시스템 호출

◆프로세스를 재사용하는 exec()

- 기존 프로세스를 종료하지 않고, 코드만 새로운 프로그램으로 변경
- 메모리와 PCB를 정리하고 새로 만드는 과정이 생략되어 (a)보다 (b)가 효율적



(a) 프로세스를 없애고 새로운 프로세스 생성

(b) exec()를 호출하여 기존 프로세스 재사용

그림 3-19 exec()와 프로세스 재사용

3. 프로세스 재사용 시스템 호출

◆프로세스를 재사용하는 exec()

코드 3-1 exec() 실행 사례

```c
#include <stdio.h>
#include <unistd.h>

void main( ) {

    printf("Before exec() \n");
    execlp("ls", "ls", "-a", (char *)NULL);
    printf("After exec() \n");
}
```

```
Before exec()
.  ..  a.out  main.c      <- ls -a 실행 결과
```

3. 프로세스 재사용 시스템 호출

◆프로세스를 재사용하는 exec()

- exec()를 실행하면 PCB를 유지하면서, 코드 영역이 교체되고 데이터 영역이 리셋됨
  - PID, PPID, CPID는 유지되고 레지스터 정보, 자원 정보는 리셋됨

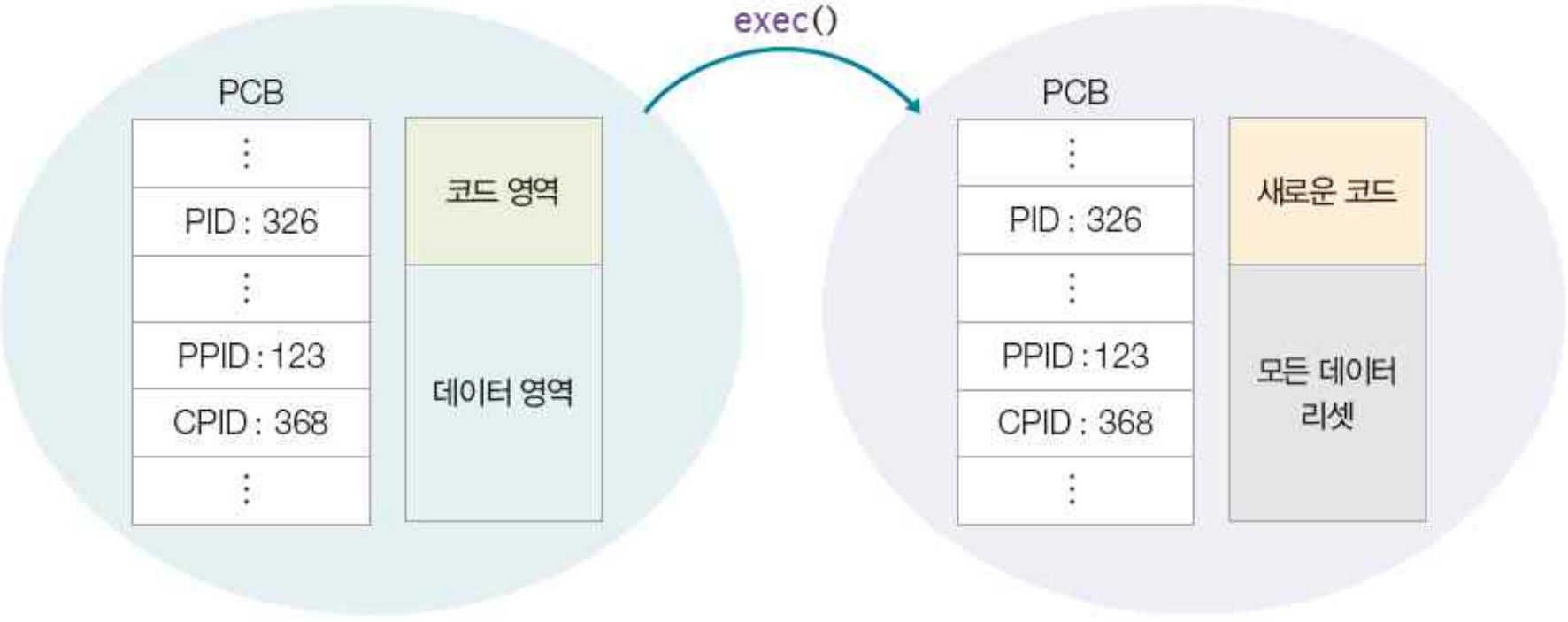


그림 3-20 exec() 사용 후 프로세스의 변화

3. 프로세스 재사용 시스템 호출

◆유닉스 셸에서 fork()와 exec()을 함께 사용하기

```
$ ps -f
UID        PID    PPID    C   STIME TTY          TIME   CMD
hs        2168    2153    0   11:59 pts/0    00:00:00   bash
hs        3030    2168    0   12:00 pts/0    00:00:00   ps -f
```

- bash 프로세스에서 fork()를 호출한 다음 exec("ps –f")를 실행했음

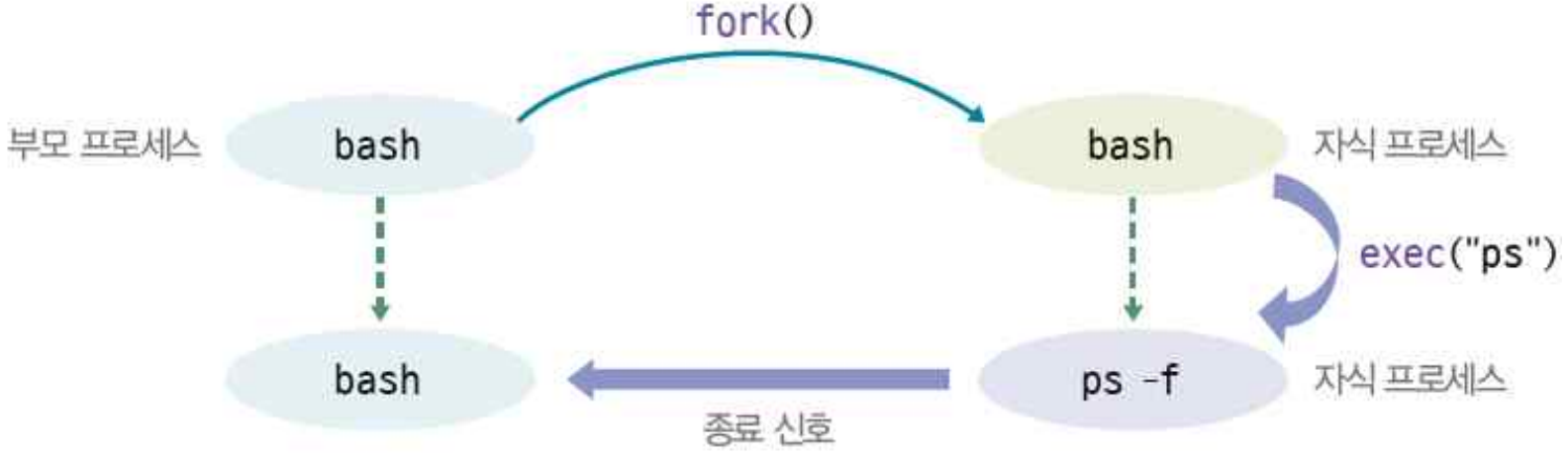


그림 3-21 fork()와 exec()을 함께 사용

4. 프로세스 동기화

◆부모가 자식 프로세스를 기다리게 하는 wait()

- 자식 프로세스가 종료 신호를 보낼 때까지 부모 프로세스가 종료하지 않고 기다림
  - 고아 프로세스

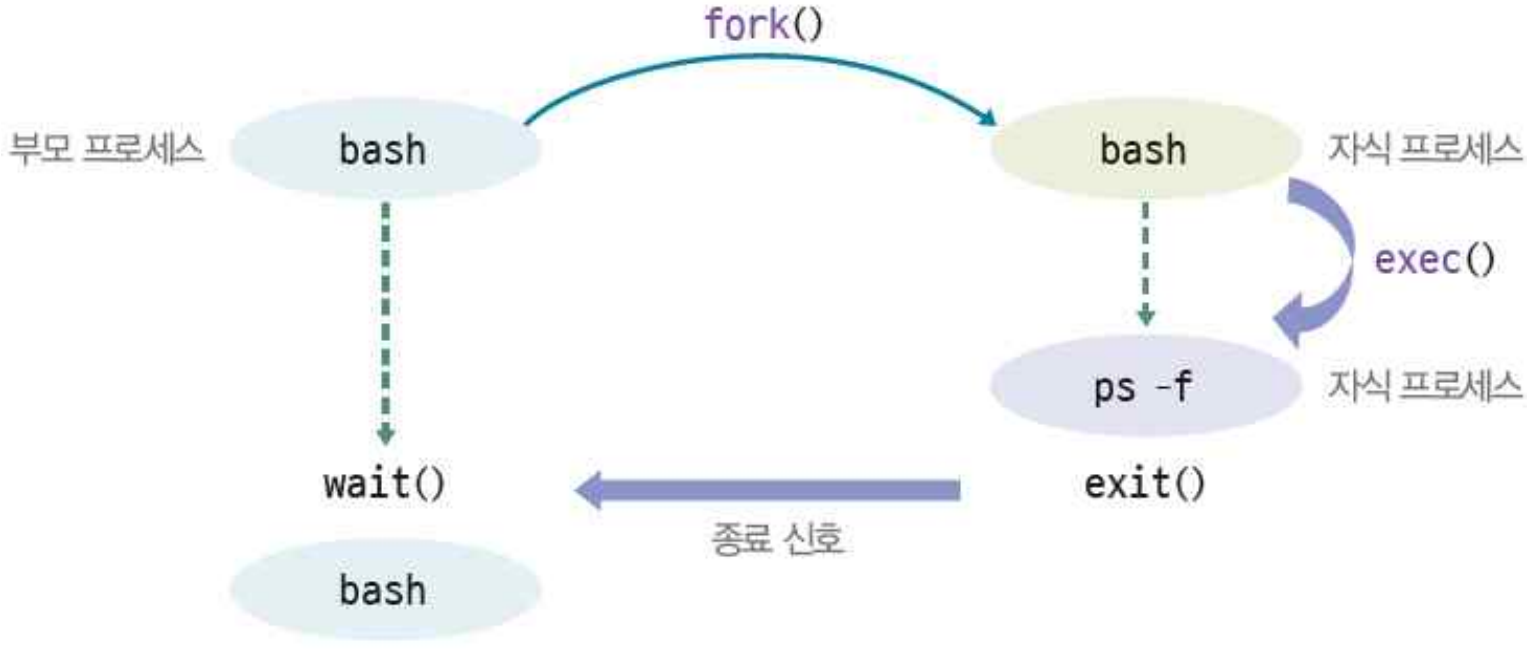


그림 3-22 wait()와 exit()의 동작

코드 3-2 fork()와 exec()의 동작

```
01  #include <stdio.h>
02  #include <unistd.h>
03  #include <sys/wait.h>
04
05  int main( ) {
06      int pd;
07
08      pd = fork();                                  // 자식 프로세스를 생성한다.
09      if (pd == 0)
10          execlp("ls", "ls", "-a", (char *)NULL);   // 자식을 ls로 변경한다.
11      else {
12          printf("Parent waiting \n");
13          wait(NULL);                               // 자식을 기다린다.
14          printf("Child finished \n");
15      }
16      return 0;
17  }
```

4. 프로세스 동기화

◆부모가 자식 프로세스를 기다리게 하는 wait()

```
Parent waiting
.  ..  a.out  main.c
Child Finished
```

- 고아 프로세스 발생 시 운영체제가 init 프로세스로 입양 보냄
- 좀비 프로세스

04 [심화] 프로세스의 데이터 영역

◆데이터 영역

- 전역변수와 지역변수
- 정적 할당과 동적 할당

◆프로세스의 가변적 영역

- 힙 영역, 스택 영역은 크기가 동적으로 변함

  ↔ 코드 영역, 정적 데이터 영역은 크기가 고정되어 있고 프로세스 실행 중 유지됨
- 힙과 스택은 서로 반대 방향으로 확장

04 [심화] 프로세스의 데이터 영역

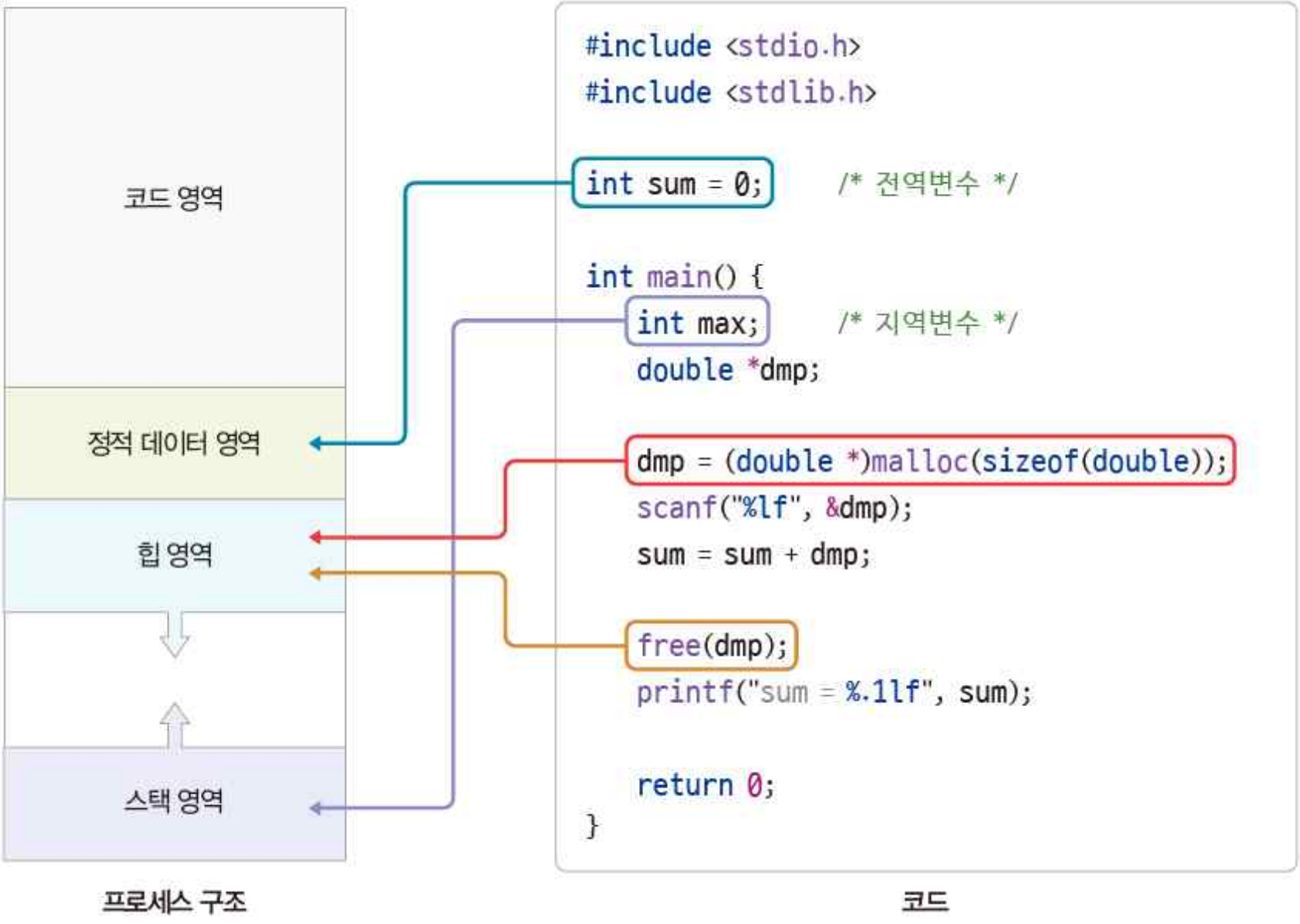


그림 3-23 프로세스의 상세 구조

04 [심화] 프로세스의 데이터 영역

◆프로세스의 가변적 영역

- 함수 호출할 때 지역변수와 복귀 주소를 스택에 저장하고, 함수 실행이 끝나면 제거
  - PUSH: 스택에 데이터 넣기, POP: 스택에서 데이터 빼내기

```
01  mul(int e, int f) {
02      printf("mul %d\n", e * f);
03  }
04
05  add(int c, int d) {
06      mul(c, d);
07      printf("add %d\n", c + d);
08  }
09
10  void main() {
11      int a = 1, b = 2;
12
13      add(a, b);
14      printf("main \n");
15  }
```

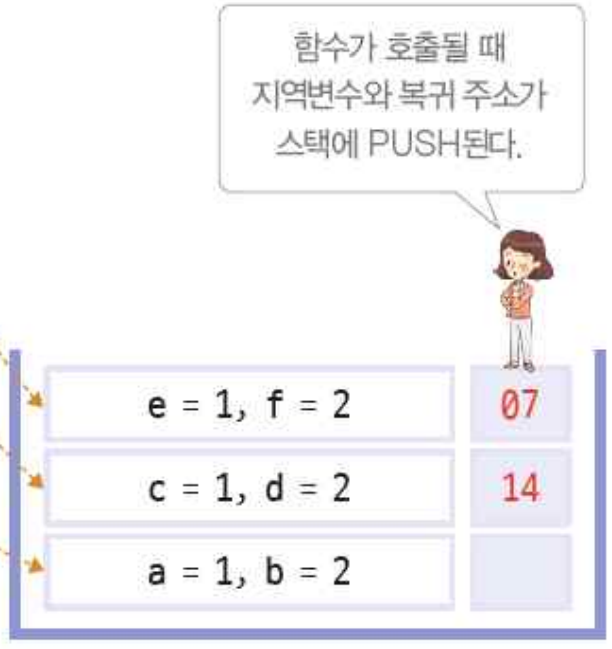


그림 3-24 함수 호출과 스택

Thank You!

Copyright© 2025 Hanbit Academy, Inc.
All rights reserved.